

ECEN 4243: Computer Architecture

Final Lab Project:

Kurt Sewell, A20340592

Matthew Brue, A20361976

Objective:

The goal of this project was to learn about and utilize page tables to implement virtual memory. Virtual memory works by creating an illusion of a large, contiguous memory independent of the actual RAM available. It works by deriving the actual physical addresses from virtual addresses that the program uses. The translation between virtual address and physical address is done in blocks of fixed size called pages. The use of a page table stores the mapping of the pages between the physical and virtual addresses and is used by the memory management unit. When a program accesses a virtual address, the memory management unit looks at the page table to find the corresponding physical address. If the page is currently not in physical memory, the operating system will load the page from secondary storage into RAM. This gives more freedom to programs to use more memory than physically available.

Implementation:

Our code works by setting up Physical Memory Protection in machine mode to allow unrestricted memory access. Then we switch to supervisor mode using the mstatus and mepc registers. In order to enable virtual memory, we then configure the satp register to

hold the root address of our page table. Next, we store the value 42 into the virtual address 0x90000300 which gets pointed to the physical address 0x80000300 through the page table.

Results:

In order to ensure that we successfully created a working page table, we first ran the program through a simulation in sail. There we verified that the L1 and L0 page tables pointed to the correct physical addresses (Figure 1) and checked to make sure that the simulation successfully completed (Figure 2). Then we ran it through the spike simulator to ensure that the value 42 did get stored to the physical 0x80000300 address (Figure 3). Finally we simulated the waveforms in QuestaSim and saw that the Program Counter jumped from physical to virtual space (Figure 4) and that the physical and virtual addresses were being properly translated (Figure 5).

Conclusion:

This project helped us gain an understanding of how to use page tables to implement virtual memory. We learned about how different levels of privilege can give us access to different commands and levels of memory. We also confirmed that programs can operate in a virtual memory space, which allows programs to use more memory than the computer may have.

Appendix I:

```
[28] [S]: 0x80000056 (0x0062A023) sw t1, 0x0(t0)
mem[R,0b001000000000000000001000100100000000] -> 0x20004401
mem[R,0b001000000000000000001000100000000000] -> 0x200000CF
mem[0b00100000000000000000000000001100000000] <- 0x0000002A
mem[R,0b001000000000000000001000100000000000] -> 0x20004401
mem[R,0b001000000000000000001000100000000000] -> 0x200000CF
mem[X,0b001000000000000000000000000000001011010] -> 0x0297
mem[X,0b001000000000000000000000000000001011100] -> 0x0000
```

Figure 1. Screenshot showing that the L1 and L0 page tables are writing to the correct physical address

```

[30] [S]: 0x8000005E (0x01228293) addi t0, t0, 0x12
x5 <- 0x8000006C
mem[X,0b001000000000000000000000000000001100010] -> 0x0337
mem[X,0b001000000000000000000000000000001100100] -> 0x1000
[31] [S]: 0x80000062 (0x10000337) lui t1, 0x10000
x6 <- 0x10000000
mem[X,0b001000000000000000000000000000001100110] -> 0x929A
[32] [S]: 0x80000066 (0x929A) c.add t0, t1
x5 <- 0x9000006C
mem[X,0b001000000000000000000000000000001101000] -> 0x8282
[33] [S]: 0x80000068 (0x8282) c.jr t0
mem[R,0b001000000000000000001000100100000000] -> 0x20004401
mem[R,0b001000000000000000001000100000000000] -> 0x200000CF
mem[X,0b001000000000000000000000000000001101100] -> 0x0293
mem[X,0b001000000000000000000000000000001101110] -> 0x02A0
[34] [S]: 0x9000006C (0x02A00293) addi t0, zero, 0x2a
x5 <- 0x0000002A
mem[X,0b001000000000000000000000000000001110000] -> 0x0497
mem[X,0b001000000000000000000000000000001110010] -> 0x0000
[35] [S]: 0x90000070 (0x00000497) auipc s1, 0x0
x9 <- 0x90000070
mem[X,0b001000000000000000000000000000001110100] -> 0x8493
mem[X,0b001000000000000000000000000000001110110] -> 0x0204
[36] [S]: 0x90000074 (0x02048493) addi s1, s1, 0x20
x9 <- 0x90000090
mem[X,0b001000000000000000000000000000001111000] -> 0x4285
[37] [S]: 0x90000078 (0x4285) c.li t0, 0x1
x5 <- 0x00000001
mem[X,0b001000000000000000000000000000001111010] -> 0xA023
mem[X,0b001000000000000000000000000000001111100] -> 0x0054
[38] [S]: 0x9000007A (0x0054A023) sw t0, 0x0(s1)
htif[0b0010000000000000000000000000000010010000] <- 0x00000001
mem[X,0b001000000000000000000000000000001111110] -> 0xA223
mem[X,0b0010000000000000000000000000000010000000] -> 0x0004
[39] [S]: 0x9000007E (0x0004A223) sw zero, 0x4(s1)
htif[0b0010000000000000000000000000000010010100] <- 0x00000000
htif-syscall-proxy cmd: 0x000000000001
SUCCESS

```

```
(spike)
core 0: 0x80000042 (0x0062e2b3) or      t0, t0, t1
(spike)
core 0: 0x80000046 (0x18029073) csrw    satp, t0
(spike)
core 0: 0x8000004a (0x900002b7) lui     t0, 0x90000
(spike)
core 0: 0x8000004e (0x30028293) addi    t0, t0, 768
(spike)
core 0: 0x80000052 (0x02a00313) li      t1, 42
(spike)
core 0: 0x80000056 (0x0062a023) sw      t1, 0(t0)
(spike)
core 0: 0x8000005a (0x00000297) auipc   t0, 0x0
(spike) mem 0x80000300
0x0000002a
(spike)
```

	InstrValidM	1'h1
	PCM	32'h80000066
	InstrM	32'h0062a023
	InstrName	SW
	IEUAdM	32'h90000300
	Isu	(Isu)
	ReadDataM	64'h0000000000000000
	WriteDataM	32'h0000002a
	FWriteDataM	64'h0000000000000000
	ReadDataWordMaxM	64'h0000000000000000
	stalls	(stalls)

Figure 4. Screenshot showing PC jumping from 80000000 to 90000000

Cache SRAM read		(Cache SRAM read)									
dtb		(dtb)									
VAddr	3276001000	80010900					80011000			90000300	
EffectivePrivilegeMode	2h1	1									
PTE	32760004401	00000000				20004401			200000cf		00000000
HiPageType	2h0	0									
Translate	1h0										
DisableTranslation	1h1										
TBMMiss	1h0										
TBMM	1h0										
PhysicalAddress	347600011000	080010900				080011000			080000300		080000300
faults		(faults)									
TBPageFault	1h0										
LoadAccessFaultM	1h0										
StoreAccessFaultM	1h0										

Figure 5. Screenshot showing the translation between virtual and physical memory addresses